

README — representation extraction

This runs a contextual model on the tokenized sample and turns it into vectors: one vector per unit, and — for a target word — a **contour**, the matrix of all its occurrences. It can also produce a quick **static** vector for a word straight from the model's embeddings. This is where the model's full weights are actually executed (not just the tokenizer).

The contextual flow is: **fit the text to the model** → **run the model** → **compose one vector per unit** → **assemble the contour**, recording every decision.

This folder contains:

File	What it is for
<code>fragmentation.py</code>	Cuts texts that are longer than the model's token limit.
<code>model_runner.py</code>	Runs the model in batches and extracts per-token vectors from the chosen layer(s).
<code>aggregation.py</code>	Composes one vector per unit from its token vectors.
<code>contour.py</code>	Assembles the occurrences × dimensions matrix for a target word.
<code>static_vector.py</code>	A word's context-free vector from the model's embeddings.
<code>tracelog.py</code>	The audit trail (a copy lives here so the folder is self-contained).
<code>exe.py</code>	The interactive program that runs everything here.

You need `pip install transformers torch numpy`. Running the model downloads the model's full weights the first time and is heavier than the earlier steps (slower, more memory).

It reads the tokenized sample (`model_tokens.json`) produced by the text-preparation-and-tokenization folder, found automatically in the sibling folder.

1. `fragmentation.py` — fitting long texts to the model

Motivation. A model accepts a limited number of tokens (e.g. 512 for BERT). Longer units must be cut, and how you cut depends on the unit type.

- **Segment units** (a target word with context): the word must never be lost or split off, so it takes a single window **centered on the segment**, with as much context as fits on each side.

- **Sentence / document units** (the whole text is the observation): the generic strategies apply:
 - **truncate** — keep the first tokens and drop the rest. Fastest, but loses everything past the limit. Use it when the start of the text is enough, or just to test quickly.
 - **fixed windows** — split into back-to-back blocks with no overlap. Covers the whole text without repeating anything, but a word sitting on a block boundary gets its context split across two blocks. Use it when you want full coverage and overlap doesn't matter.
 - **sliding windows** — overlapping blocks. Each block shares some tokens with the next, so words near a boundary still appear whole in at least one block. Best coverage and context, at the cost of more blocks to process. Use it when context quality matters (the usual choice for documents).

Two parameters control sliding windows:

- **window size** — how many tokens each block holds (usually the model's limit, e.g. 512).
- **stride** — how far the window advances each step. Smaller stride = more overlap (window 512 / stride 256 = 50% overlap); stride equal to the window = no overlap (the same as fixed windows). Smaller stride gives safer coverage but produces more blocks.

Function

- `fragment_units(units, max_tokens, strategy, window, stride, save_to)` — Fragments only the units that exceed `max_tokens`. Short ones pass through unchanged. Each resulting fragment records how it was cut and the token range it covers. Saving is optional.
-

2. `model_runner.py` — running the model

Motivation. This executes the model and gets, per token, a vector from the chosen layer(s). It handles the practical side of feeding text to a model:

- **Batching:** units are processed in groups of `batch_size` at a time (typically 8-64). This is purely about speed vs. memory: a larger batch runs faster but uses more memory, and too large a batch runs out of memory. Lower it if you hit memory errors; raise it if you have memory to spare and want more speed. It does not change the results.
- **Padding + attention masks:** within a batch, shorter sequences are padded to a common length and a mask marks real text vs. padding, so padding does not affect the output. (The padding is dropped again afterwards.) For models with no padding token (like GPT-2) the end-of-sequence token is used as padding automatically.
- **Layer choice:** a model has many layers, and they encode different things — lower layers capture surface features (word form), middle ones syntax, upper ones meaning.

Two options:

- **last layer** — the final layer, most tied to the training objective. A simple, common default.
- **average of the last N layers** — averages the top N layers, which tends to give a smoother, often more robust meaning representation than any single layer. Use it when you want a stable semantic vector; pick the single last layer when you want the most task-tuned one or to keep it simple. (For BERT the last or last few work well; for GPT, the last.)

Functions

- `load_model(model_name)` — Loads the model's full weights (downloads them the first time), set up to expose all layers.
 - `load_tokenizer_for_model(model_name, model)` — Loads a tokenizer ready for batching, fixing the missing padding token for models like GPT-2.
 - `run_model(units, model_name, layer_mode, n_last, batch_size, save_to)` — Runs the model and attaches `token_vectors` (one per token) to each unit. Saving the per-token vectors is **optional** because that file can be very large.
-

3. `aggregation.py` — one vector per unit

Motivation. The model gives one vector per token, but a unit usually spans several tokens, so they must be composed into one vector. Two strategies:

- **Mean pooling:** average the token vectors.
- **Representative token:** take one token's vector ([CLS] for bidirectional models, the last token for autoregressive ones).

Which tokens are used depends on the unit type: for a **segment**, only the target word's tokens; for a **sentence** / **document**, all of them.

Function

- `aggregate_units(units, strategy, representative, save_to)` — Adds a final `vector` to each unit. Units whose segment could not be aligned get no vector (and are flagged). The saved file keeps one vector per unit and drops the heavy per-token vectors, so it stays small.
-

4. `contour.py` — the contour of a word

Motivation. For a target word, the interesting object is usually not one isolated vector but the word's behaviour **across its contexts**. The contour gathers every occurrence's vector

into one matrix:

```
rows    = occurrences of the word (one per context)
columns = dimensions of the representation
```

This matrix is the word's semantic portrait — what lets you later compare contexts, measure the spread of senses, or compare the word between conditions. Each row keeps its source (article, context) so rows can be grouped afterwards. If the same occurrence shows up in more than one unit (for example because a long text was split into overlapping windows), it is counted **once** (rows are deduplicated by document and position). Sentence / document units have no single repeated word, so a contour does not apply to them.

Function

- `build_contour(units, word, save_to)` — Assembles the matrix from the units that have a vector, deduplicating repeated occurrences, keeping per-row metadata, and saving it with its shape.
-

5. `static_vector.py` — a word's static vector

Motivation. A third kind of representation, alongside the contextual vectors here and LSA elsewhere. It reads a word's vector directly from the model's **input embedding** table — the vectors the model uses before any context is applied. So it is one context-free vector per word, cheap to get (no need to run the model on text). Useful as a simple baseline. If the word splits into several sub-word tokens, their vectors are averaged.

Function

- `get_static_vector(term, model_name, normalize)` — Returns the term's static vector from the model's embeddings, optionally L2-normalized.
-

6. The exe (`exe.py`) — the interactive program

Motivation. Ties everything together: finds a tokenized sample, and builds the representation you choose, saving the outputs and recording every decision.

Every question, in order

1. **Which tokenized sample** — a menu of the `model_tokens.json` files found in this folder's `output/` or a sibling folder's `output/`. (The **model is not asked**: it is taken automatically from the tokenized sample — the same one used to tokenize — since running a different model on those tokens would be incorrect. It is shown, not chosen.)

2. Which representation:

- **contextual** — runs the model on each occurrence's context (the full flow below);
- **static** — the short path: one context-free vector per target word from the model's embeddings (asks whether to L2-normalize), saved to `static_vectors.json`.

For the **contextual** path it then asks:

3. **Fragmentation** — the model's token limit is read automatically (not asked); you choose the strategy (sliding / fixed / truncate) for sentence/document units (segment units are always centered on the word), the window and stride for sliding, and whether to save the fragmented units.
4. **Run the model** — which layer(s) (last / average of last N), batch size, and **whether to save the per-token vectors**.
5. **Aggregation** — mean pooling or representative token (and which token).

It then builds the **contour** automatically when the units are segments.

What gets saved

- **Always:** the aggregated vectors (`aggregated.json`) and, for segments, the contour (`contour.json`) — both reasonable in size. For the static path, `static_vectors.json`.
- **Optional:** the fragmented units (`fragmented.json`) and the per-token vectors (`token_vectors.json`, the heavy step).
- **Always:** the tracelog (`tracelog_part3.json`), with the model, layer choice, aggregation, fragmentation, and any duplicates removed.

The two optional files are intermediate results, off by default because they are usually not needed for the analysis:

- `fragmented.json` records how each long text was cut (which token range each fragment covers, how it was split). Useful to *audit the cut* — e.g. to check the target word stayed inside its centered window, or how many fragments a long document produced. Skip it if your texts are short (nothing is cut) or you only care about the final vectors.
- `token_vectors.json` holds the model's raw output: one vector for *every token* of every unit (the step before they are combined into one vector per unit). It is what aggregation reads to build the final vectors. Saving it lets you inspect or re-aggregate the raw per-token output later without re-running the model — but it is large (hundreds of numbers per token, for every token), so it is off by default. Skip it unless you specifically need the raw token-level vectors.

Outputs go to `output/<run>/` inside this folder.

Status

The contextual flow, the static path, and the exe are all in place and work. What is **not built yet**: the **anisotropy adjustments** (checking and correcting the geometry of the vectors), planned as a later addition between aggregation and the contour.

Turning a vector or a contour into a final research measure is deliberately out of scope here: that depends on the specific research question and is designed separately.

How it chains together

```
model_tokens.json    (from the tokenization folder)
    |
    └─ static        →  output/<run>/static_vectors.json    (one vector per
word)
    |
    └─ contextual:
        fragment      →  fit each unit to the model's token limit
            |
            run model  →  per-token vectors from the chosen layer(s)
(optional save)
            |
            aggregate  →  one vector per unit
(saved)
            |
            contour    →  occurrences × dimensions matrix for the word
(saved)
            |
            tracelog   (every decision, with its reason)
```